INTERNSHIP REPORT

Full Stack Development of KioMeet: A Secure Video Conferencing and Technical Interview Platform

by

Aman Verma
Entry No.: 23BCS012

Submitted in partial fulfilment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING



SHRI MATA VAISHNO DEVI UNIVERSITY, KATRA
(School of Computer Science & Engineering)
JAMMU & KASHMIR – 182 320

Aug 2026

CERTIFICATE

This is to certify that Mr. **Aman Verma** (Entry No. 23BCS012) has successfully completed Industrial Training during the period from **12 June 2026** to **7 August 2026** at **IndiWebPros** as a Partial Fulfilment of the Degree of Bachelor of Technology in Computer Science and Engineering. He was trained in the field of **Full Stack Development** under the supervision of the Program Management Office (PMO), and satisfactorily completed his training, contributing to the design and implementation of modules for *KioMeet*, a secure cross-platform video conferencing and technical-interview platform.

Signature & Seal of Training Manager

Mohan
Founder & Director
IndiWebPros
Ref: IWP-STU-2026-0904

DECLARATION

I, **Aman Verma**, Entry No. 23BCS012, hereby declare that the work presented in this Internship Report titled *"Full Stack Development of KioMeet: A Secure Video Conferencing and Technical Interview Platform"* is an authentic record of my own work carried out during my internship at **IndiWebPros** from **12 June 2026** to **7 August 2026**, under the supervision of the company's Program Management Office and the guidance of my institute mentor. This work has not been submitted elsewhere for the award of any other degree or diploma.

Date: October 9, 2026

Aman Verma
23BCS012

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to **Mr. Mohan**, Founder & Director of **IndiWebPros**, for giving me the opportunity to intern with the organisation and for his encouragement throughout the engagement. I am equally thankful to the **Program Management Office (PMO)** at IndiWebPros for their constant guidance, timely feedback during weekly evaluations, and for structuring a learning environment that allowed me to contribute meaningfully to a live technical module.

Finally, I thank my family, friends, and the faculty of the School of Computer Science & Engineering for their continued support and encouragement throughout the duration of this internship.

Aman Verma

COMPANY PROFILE

IndiWebPros is a community-driven technology startup focused on technical excellence and collaborative growth. The organisation runs structured, fully virtual internship programmes in which interns are onboarded into a Program Management Office (PMO) that supervises live technical modules, community sprints, and a capstone project aligned to the intern's chosen skill track.

Website: `https://www.indiwebpros.in`

Contact: internships@indiwebpros.in

Engagement model: Virtual / remote internship with weekly evaluation

Primary skill tracks offered: Full Stack Development (the track undertaken for this report), among other technical domains

Supervising authority: Program Management Office (PMO)

Leadership: Mohan, Founder & Director

As per the terms of engagement (Offer Ref. IWP-STU-2026-0904), interns at IndiWebPros work under non-disclosure terms: all software, documentation, and intellectual property produced during the internship remain the property of IndiWebPros, and interns are expected to maintain confidentiality of non-public information encountered during their engagement. Certification of the internship is contingent on successful completion of assigned tasks and adherence to the organisation's code of conduct.

ABSTRACT

This report documents an eight-week internship (12 June 2026 – 7 August 2026) completed at **IndiWebPros** under the Full Stack Development track, supervised by the company's Program Management Office. The capstone project undertaken was **KioMeet**, a native, cross-platform (Windows and macOS) video conferencing application purpose-built for team calls and technical interviews. KioMeet pairs a native desktop client – built without a browser or Electron layer – with a Node.js/Express backend, a LiveKit-compatible WebRTC media layer, PostgreSQL for persistent data, and Redis for ephemeral room state.

The internship work centred on the backend and full-stack concerns of the platform: JWT-based authentication with rotating refresh tokens, tier-gated access control, meeting scheduling with calendar synchronisation, team management, and the real-time signalling that supports in-meeting features such as chat, polls, and a Q&A board. A particular contribution was the backend session logic behind KioMeet's secure interview mode – a distinguishing feature of the platform that isolates an interviewer's screen content from the shared stream, so that only a single designated window reaches the candidate.

This report presents the requirement specification, system design, testing approach, and results obtained during the internship, along with a discussion of the skills gained and the scope for future enhancement of the platform.

Keywords: Cross-platform desktop application, Full stack development, JWT authentication, LiveKit, Node.js, PostgreSQL, Redis, Secure interview mode, Video conferencing, WebRTC.

TABLE OF CONTENTS

Table of Contents vi

List of Figures viii

List of Tables ix

Abbreviations and Nomenclature x

Chapter 1: INTRODUCTION 1
- 1.1 PURPOSE . . . 1
- 1.2 SYSTEM OVERVIEW . . . 1
- 1.3 PROBLEM STATEMENT . . . 1
- 1.4 INTERNSHIP ROLE AND SCOPE OF WORK . . . 2
- 1.5 GOAL AND VISION . . . 3
- 1.6 REPORT OUTLINE . . . 3

Chapter 2: REQUIREMENT SPECIFICATION 4
- 2.1 USER CHARACTERISTICS . . . 4
- 2.2 USE CASES (FUNCTIONAL REQUIREMENTS) . . . 4
  - 2.2.1 Use Case Diagram . . . 4
- 2.3 DEPENDENCIES . . . 4
- 2.4 PERFORMANCE REQUIREMENTS . . . 4
- 2.5 HARDWARE REQUIREMENTS . . . 5
- 2.6 CONSTRAINTS AND ASSUMPTIONS . . . 5

Chapter 3: SYSTEM DESIGN 7
- 3.1 ARCHITECTURE . . . 7
  - 3.1.1 Authentication Design . . . 7
  - 3.1.2 Tier and Billing Design . . . 7
  - 3.1.3 Secure Interview Session Design . . . 7
  - 3.1.4 Real-Time Signalling . . . 7
- 3.2 DATABASE DESIGN . . . 8
- 3.3 API DESIGN . . . 8
- 3.4 GUI / INTERFACE DESIGN . . . 8
- 3.5 PACKAGE / MODULE HIERARCHY . . . 8

Chapter 4: TESTING **9**
4.1 TEST PLAN 9
4.2 TEST REPORT 9
4.3 DEFECT LOG 9
4.4 INSTALLATION AND END-USER INSTRUCTIONS 10

Chapter 5: RESULTS AND DISCUSSION **11**
5.1 OUTCOMES ACHIEVED 11
5.2 DISCUSSION 11
5.3 SKILLS GAINED 12

Chapter 6: CONCLUSION AND FUTURE SCOPE **13**
6.1 FUTURE SCOPE 13

REFERENCES **14**

APPENDIX I **15**

APPENDIX II: LIST OF SOFT OUTPUTS **16**

LIST OF FIGURES

1 KioMeet layered system architecture 2
2 Context-level data-flow diagram (DFD) of the KioMeet system 2

LIST OF TABLES

1 Principal use cases of the backend module 5
2 Weekly progress during the internship 11

ABBREVIATIONS AND NOMENCLATURE

API	Application Programming Interface
AWS	Amazon Web Services
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity-Relationship
GUI	Graphical User Interface
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
OAuth	Open Authorization
PMO	Program Management Office
RLS	Row-Level Security
S3	Simple Storage Service (AWS)
SFU	Selective Forwarding Unit
SPA	Single-Page Application
SQL	Structured Query Language
TTL	Time to Live
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
WebRTC	Web Real-Time Communication

CHAPTER 1: INTRODUCTION

With the rise of remote hiring and distributed teamwork, dependable video conferencing has become a core part of everyday technical work. Most general-purpose meeting tools are browser- or Electron-based, which limits access to native operating-system capabilities and offers no reliable way to prevent private material on an interviewer's screen from leaking into a shared video stream during a technical interview. This chapter introduces the internship engagement, the host organisation, and the capstone project undertaken.

1.1 PURPOSE

The purpose of this internship was to gain hands-on, full-stack software development experience in a live, production-style codebase, under the mentorship of an industry Program Management Office, while contributing a substantial module to a real product – **KioMeet** – as the capstone project required under the Full Stack Development track.

1.2 SYSTEM OVERVIEW

KioMeet is organised in three cooperating layers:

- **Native desktop client** (Windows and macOS) – owns media capture, GPU-accelerated rendering, the secure interview window, and the user interface.
- **Real-time media layer** – a LiveKit-compatible WebRTC server that publishes and subscribes audio, video, and screen-share tracks.
- **Backend token server** (Node.js + Express) – issues short-lived access tokens, manages rooms, enforces user tiers, schedules meetings, synchronises calendars, and tracks recordings, backed by PostgreSQL for persistent data and Redis for ephemeral room state.

Figure 1 illustrates this layered architecture.

1.3 PROBLEM STATEMENT

The problem addressed by KioMeet is to design and build a native, cross-platform (Windows and macOS) video conferencing platform that supports both standard team calls and technical interviews, in which an interviewer's private material is provably isolated from the shared video stream, while delivering reliable multi-participant audio/video, screen sharing, recording, scheduling, and moderation through a secure, tier-gated backend – without the overhead and capability limits of browser- or Electron-based solutions.

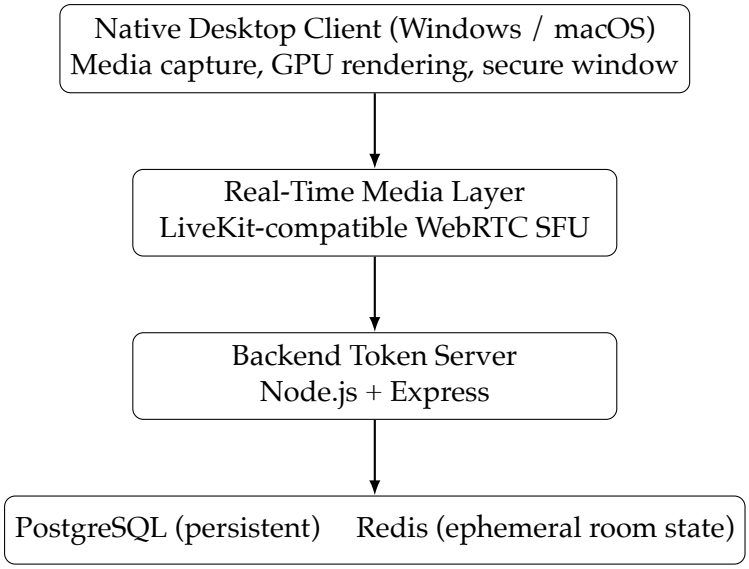


Figure 1: KioMeet layered system architecture

Figure 2 presents a context-level data-flow diagram (DFD) of the problem. It shows the external entities that interact with the KioMeet system and the principal data flows between them, scoping the boundary of the module undertaken during the internship.

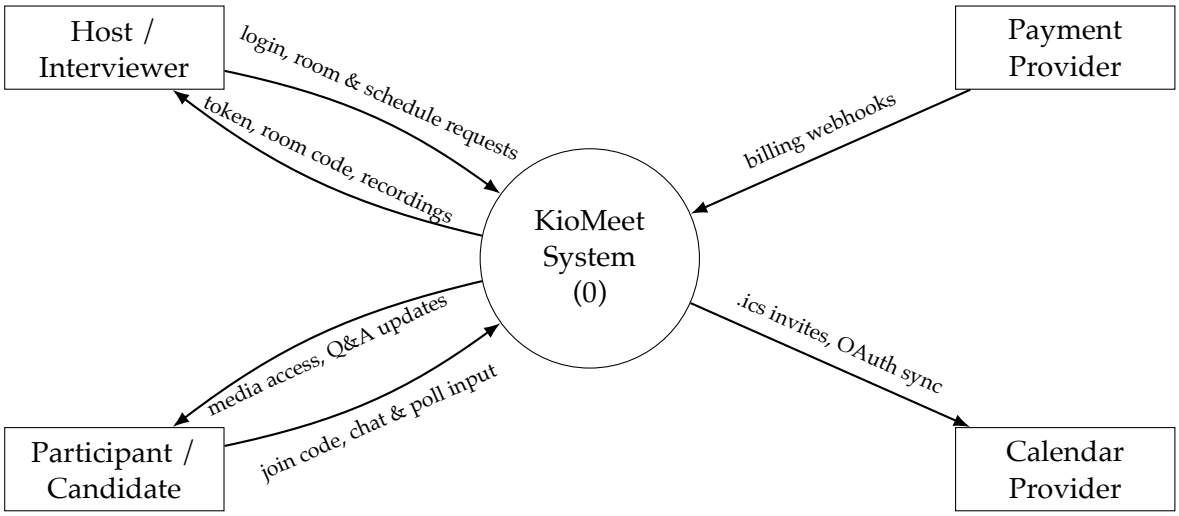


Figure 2: Context-level data-flow diagram (DFD) of the KioMeet system

1.4 INTERNSHIP ROLE AND SCOPE OF WORK

Under the Full Stack Development track and the guidance of the PMO, the scope of work carried out during the internship (12 June – 7 August 2026) included:

- Contributing to the Node.js/Express backend (`token-server/`): authentication, scheduling, teams, and tier/billing modules.
- Building the backend session logic behind KioMeet's secure interview mode –

creating and permission-gating a locked `secure-interview` session type that the client uses to restrict the shared view to a single designated window.

- Implementing the real-time signalling (WebSocket channels) that delivers in-call chat messages, live poll results, and Q&A updates to every participant instantly.
- Working with PostgreSQL schema migrations and Redis-backed ephemeral room state.
- Assisting with the `launch-web` marketing site's waitlist API and admin panel.
- Participating in weekly PMO evaluation sprints and community code reviews.

1.5 GOAL AND VISION

The goal of the internship was twofold: to deliver a working, tested contribution to KioMeet's backend within the eight-week engagement, and to build practical competence in authentication design, relational data modelling, and real-time systems – skills directly transferable to production full-stack engineering roles.

1.6 REPORT OUTLINE

The remainder of this report is organised as follows. Chapter 2 specifies the requirements of the module undertaken. Chapter 3 presents the system design. Chapter 4 describes the testing approach and results. Chapter 5 discusses the outcomes achieved during the internship. Chapter 6 concludes the report and outlines scope for future work.

CHAPTER 2: REQUIREMENT SPECIFICATION

This chapter specifies the functional and non-functional requirements of the KioMeet backend module undertaken during the internship.

2.1 USER CHARACTERISTICS

KioMeet serves two primary user classes relevant to the backend module:

- **Hosts / Interviewers** – create rooms, schedule meetings, manage recordings, and moderate participants.
- **Participants / Candidates** – join rooms via a 6-character code, respond to lobby / password gates, and participate in chat, polls, and Q&A.

Both classes authenticate via the JWT-based auth flow, and their accessible features are gated by subscription tier (free, pro, hiring).

2.2 USE CASES (FUNCTIONAL REQUIREMENTS)

Table 1 summarises the principal use cases implemented or touched during the internship.

2.2.1 Use Case Diagram

A simplified use-case view of the authentication and room-management flow is as follows: an *Actor (Host/Participant)* interacts with *Register/Login*, *Create or Join Room*, and *Schedule Meeting*, each of which depends on the *Authenticate* use case for authorization.

2.3 DEPENDENCIES

The module depends on: Node.js 20+, Express, PostgreSQL 15+, Redis 7+, a LiveKit-compatible WebRTC server, and third-party calendar OAuth providers (Google, Microsoft/Outlook).

2.4 PERFORMANCE REQUIREMENTS

- Access tokens must expire within 15 minutes; this bounds the maximum staleness of a user's tier after a billing change.
- Room lookups and join-code validation must complete within Redis's typical sub-millisecond read latency to avoid perceptible join delay.

Table 1: Principal use cases of the backend module

Use Case	Description
Register / Login	User registers or logs in; receives a short-lived JWT access token and a rotating refresh token.
Refresh Session	Client exchanges a valid refresh token for a new access token; tier is re-read from the database on every refresh.
Create Room	Host creates a call or interview room; a 6-character join code is generated with a 24-hour expiry in Redis.
Join Room	Participant joins via room code, optional password, and lobby approval.
Schedule Meeting	Host schedules a meeting with invitees, who receive an `.ics` email invite; synced to Apple/Google/Outlook calendars.
Manage Team	Team owner/admin invites members, assigns roles, and manages team membership.
Start Secure Interview	Host starts a locked `secure-interview` session; the backend restricts the session to a single shared-window grant that the client enforces on screen.
Live Chat / Poll / Q&A	Backend relays chat messages, poll launches/results, and Q&A items over a per-room WebSocket channel so every participant updates in real time.
Enforce Tier Limits	Backend enforces per-tier limits on recording mode, poll count, and room capacity.

- The system must support rooms of up to 50 participants without degrading signalling responsiveness.

2.5 HARDWARE REQUIREMENTS

- Development: a standard laptop/desktop capable of running Node.js, PostgreSQL, and Redis locally (minimum 8 GB RAM recommended).
- Deployment: a server or cloud instance able to host the Express backend, PostgreSQL, and Redis, reachable by the LiveKit WebRTC server and the native desktop clients.

2.6 CONSTRAINTS AND ASSUMPTIONS

- All financial/tier state is authoritative only in `users.tier` in PostgreSQL; the client is never trusted to assert its own tier.
- The internship module could be developed and demonstrated against a local PostgreSQL/Redis instance; access to the production LiveKit deployment was out of scope for the internship.
- Development followed the engagement's confidentiality terms (Offer Ref. IWP-STU-2026-0904): internal implementation detail beyond what is already documented in the project's public-facing materials is presented here only at a level suitable for an academic report.

CHAPTER 3: SYSTEM DESIGN

3.1 ARCHITECTURE

The backend follows a layered Express architecture with a clear separation of concerns: `auth.js` handles login, logout, and token issuance; `billing.js` handles subscription and tier state; `scheduling.js` and `calendar.js` handle meeting scheduling and OAuth calendar sync; `teams.js` handles team management; and a shared `middleware` layer (`requireAuth`, `requireTier`) reads state and makes no business decisions of its own.

3.1.1 Authentication Design

A two-token model is used: a short-lived (15-minute) JWT access token signed with HS256, and a long-lived (30-day) opaque refresh token stored only as a SHA-256 hash in the database. Refresh requests are processed inside an atomic transaction with family-based theft detection: if a revoked token is re-presented, the entire token family is revoked and the session is forced to re-authenticate.

3.1.2 Tier and Billing Design

Subscription tier changes follow an explicit state machine (`none` → `trialing`/`active` → `past_due`/`canceled`, with recovery and chargeback paths) and are driven exclusively by signed payment-provider webhooks, never by client input. All tier changes are atomic and accompanied by an append-only audit record in `user_tier_history`.

3.1.3 Secure Interview Session Design

A room may be started in a locked `secure-interview` mode. On session start, the backend issues the host a single *share grant* scoped to one window handle; the client enforces the actual window-level capture restriction, but it is the backend that creates the session record, marks it `locked`, and rejects any attempt (by either participant) to attach a second shared surface to that session. This keeps the isolation guarantee backend-verifiable rather than resting purely on client-side behaviour.

3.1.4 Real-Time Signalling

Chat, polls, and Q&A are delivered over a per-room WebSocket channel (separate from the WebRTC media path). Each event (a new chat message, a poll being launched, a vote being cast, a question being asked or marked answered) is validated against the sender's role and tier, persisted, and then broadcast to every connected participant in that room, so the host and all participants see the same state update within the same round trip.

3.2 DATABASE DESIGN

Persistent entities (`users`, `meetings`, `meeting_participants`, `recording_sessions`, `scheduled_me` `meeting_invitees`, `teams`, `team_members`) are modelled in PostgreSQL with explicit foreign-key relationships and a deliberate `ON DELETE` policy per relationship (e.g. `RESTRICT` on purchase history, `CASCADE` on refresh tokens). Migrations are forward-only, numbered sequentially, and tracked in a `schema_migrations` table so re-running the migration runner is safe and idempotent. Ephemeral room state (active room codes, presence) is kept in Redis with a 24-hour TTL rather than in PostgreSQL, since it does not need to survive beyond a room's lifetime.

3.3 API DESIGN

Every endpoint follows a consistent error contract – `{ error, code, requestId }` – so the client can switch on a stable machine-readable `code` (e.g. `TOKEN_EXPIRED`, `TIER_INSUFFICIENT`, `RATE_LIMITED`) rather than parsing human-readable text. Security headers (HSTS, `Cache-Control: no-store`, `X-Content-Type-Options`) are applied globally via middleware, and sensitive fields (`password`, `refreshToken`) are scrubbed before any logging.

3.4 GUI / INTERFACE DESIGN

On the client side, the relevant interfaces for the module worked on are the connection setup panel (server URL, room code entry), the schedule panel (meeting creation and invitee management), and the recording list panel. On the web side, the `launch-web` marketing site exposes a public waitlist form and a password-protected admin panel for reviewing signups, both served from the same Express process as static assets plus a JSON API.

3.5 PACKAGE / MODULE HIERARCHY

The backend (`token-server/`) is organised as a flat set of single-responsibility Express routers/modules (`auth, billing, scheduling, calendar, teams, mailer, crypto, migrate`) wired together in `index.js`, which also mounts the health check endpoint and global security middleware.

CHAPTER 4: TESTING

4.1 TEST PLAN

Testing of the module followed the project's existing conventions: unit tests for isolated logic (token signing/verification, tier-transition validation, webhook-signature checks) and integration tests against a local PostgreSQL and Redis instance for end-to-end flows (register → login → refresh → create room → join room). Where a live dependency (e.g. the production LiveKit server) was unreachable, tests were designed to skip gracefully rather than fail, consistent with the project's existing `XCTSkip`-style pattern on the native client test suite.

4.2 TEST REPORT

- Authentication flow – verified correct issuance and rejection of expired vs. invalid access tokens, and family-wide revocation on reuse of a revoked refresh token.
- Tier enforcement – verified that `requireTier` middleware fails closed (403 `TIER_INSUFFICIEN` when a user's tier is `null` or below the required rank.
- Scheduling – verified that an `.ics` invite is generated for each invitee and that a scheduled meeting appears correctly under both the host's and the invitee's upcoming-meetings list.
- Webhook idempotency – verified that replaying the same billing webhook event id a second time is a no-op (`already_processed`) and does not duplicate a tier change.
- Secure interview sessions – verified that a second share-grant request against an already-locked session is rejected, and that the session record correctly reflects `locked` state for the duration of the interview.
- Real-time signalling – verified that a chat message, poll result, or Q&A update sent by one participant is broadcast to and received by every other connected participant in the same room.

4.3 DEFECT LOG

Representative issues identified and resolved during the internship period included an edge case where a webhook received before its corresponding user record existed was

silently dropped (fixed by queuing and retrying), and a missing `Retry-After` header on one rate-limited endpoint (fixed to match the project's documented error contract).

4.4 INSTALLATION AND END-USER INSTRUCTIONS

The backend is run locally via `npm install` followed by `node migrate.js` (to apply schema migrations) and `node index.js` (or `npx nodemon index.js` for development), after configuring `token-server/.env` per `.env.example`. A healthy instance responds to `GET /health` with Redis and PostgreSQL connection status.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 OUTCOMES ACHIEVED

By the end of the internship, the assigned backend contributions were functional, tested against a local PostgreSQL/Redis environment, and reviewed by the PMO. Table 2 summarises the weekly progress over the eight-week engagement.

Table 2: Weekly progress during the internship

Week	Dates (2026)	Summary of Work
1	12 – 18 Jun	Onboarding, codebase walkthrough, environment setup (Node.js, PostgreSQL, Redis).
2	19 – 25 Jun	Studied authentication module; implemented unit tests for JWT issuance/verification.
3	26 Jun – 2 Jul	Worked on refresh-token rotation and family-based theft detection.
4	3 – 9 Jul	Contributed to scheduling module (`.ics` invites, calendar sync touchpoints).
5	10 – 16 Jul	Implemented and tested tier-enforcement middleware edge cases.
6	17 – 23 Jul	Worked on team management module (role-based permissions); built the secure-interview session/locking logic.
7	24 – 30 Jul	Implemented real-time chat/poll/Q&A signalling; integration testing across auth, scheduling, teams, and signalling modules.
8	31 Jul – 7 Aug	Bug fixes, documentation, final PMO review and demo.

5.2 DISCUSSION

The internship provided direct exposure to production-oriented backend patterns that are often only discussed abstractly in coursework: atomic multi-step database transactions, fail-closed authorization, idempotent webhook processing, and a consistent machine-readable API error contract. Working within an existing, opinionated codebase (rather than a greenfield project) also required adapting to established conventions – for instance, the project's strict separation between `auth.js` and `billing.js` – which reinforced the practical value of separation of concerns beyond its theoretical treatment in software engineering courses.

5.3 SKILLS GAINED

- Practical experience with Node.js/Express, PostgreSQL, and Redis in a real backend.
- JWT-based authentication design, including refresh-token rotation and theft detection.
- Writing and reasoning about idempotent, webhook-driven business logic.
- Designing backend-verifiable session state for a security-sensitive feature (secure interview mode), rather than relying solely on client-side enforcement.
- Building real-time, WebSocket-based features (chat, polls, Q&A) and reasoning about event ordering and broadcast correctness.
- Collaborative, review-driven development under a PMO-style supervision model with weekly evaluation.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

This eight-week internship at IndiWebPros, under the Full Stack Development track, provided practical, supervised experience contributing to KioMeet, a real cross-platform video conferencing and secure technical-interview platform. The engagement covered the full backend lifecycle relevant to the module undertaken – requirement understanding, design, implementation, and testing – and resulted in a working, reviewed contribution to the authentication, scheduling, team-management, secure-interview-session, and real-time signalling modules of the product.

6.1 FUTURE SCOPE

Future enhancements identified but out of scope for the internship period include: extending multi-track cloud recording to additional storage backends beyond S3, adding row-level security policies to more user-owned tables, and building automated end-to-end tests that exercise the full client-to-backend flow against a live LiveKit instance rather than mocked media tracks.

REFERENCES

[1] WebRTC — Official Project Documentation. https://webrtc.org

[2] LiveKit — Open-Source WebRTC SFU Documentation. https://docs.livekit.io

[3] IETF RFC 7519 — JSON Web Token (JWT).

[4] IETF RFC 8825 — Overview: Real-Time Protocols for Browser-Based Applications (WebRTC).

[5] Express.js — Official Documentation. https://expressjs.com

[6] PostgreSQL 15 — Official Documentation. https://www.postgresql.org/docs/15/

[7] Redis — Official Documentation. https://redis.io/docs/

[8] IndiWebPros, *Offer of Internship with IndiWebPros*, Ref. IWP-STU-2026-0904, 07 June 2026 (internal document).

APPENDIX I

Sample backend health-check response (`GET /health`):

```
{
  "status": "ok",
  "redis": "connected",
  "postgres": "connected"
}
```

APPENDIX II: LIST OF SOFT OUTPUTS

1. Soft copy (PDF) of this report.
2. Offer-of-internship letter from IndiWebPros (Ref. IWP-STU-2026-0904).
3. Relevant source-code modules contributed to during the internship.
4. Weekly progress notes corresponding to Table 2.